

## **EXP NO: 2**

# **MULTI-LAYER PERCEPTRON (MLP) FOR CLASSIFICATION**

## **AIM**

To develop a Multi-Layer Perceptron (MLP) for classification using the Iris dataset and evaluate its performance using different hidden layer configurations and activation functions.

## **OBJECTIVES**

1. To understand the structure and working of an MLP.
2. To experiment with different numbers of hidden layers.
3. To compare different activation functions such as ReLU, Tanh and Sigmoid.
4. To train the MLP using the Iris dataset.
5. To evaluate the model using accuracy and loss.
6. To visualize the training and validation performance.
7. To predict the class of a new Iris flower.

## **SOFTWARE REQUIREMENTS**

- Google Colab
- Python 3.x
- TensorFlow / Keras
- NumPy
- Scikit-learn
- Matplotlib

## **DATASET**

### **Iris Dataset**

The Iris dataset contains 150 samples with four features:

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width

The three target classes are:

- Setosa
- Versicolor
- Virginica

## **THEORY**

A Multi-Layer Perceptron is a feedforward artificial neural network containing an input layer, one or more hidden layers and an output layer.

For the Iris dataset:

**Input Layer → Hidden Layer(s) → Output Layer**

- Input layer: 4 neurons
- Hidden layers: Different configurations are tested
- Output layer: 3 neurons
- Output activation: Softmax

The MLP is trained using the **Adam optimizer** and **categorical cross-entropy loss**.

## ALGORITHM

1. Import the required Python libraries.
2. Load the Iris dataset.
3. Separate input features and target labels.
4. One-hot encode the target labels.
5. Standardize the input features.
6. Split the dataset into training and testing sets.
7. Create an MLP model.
8. Test different hidden layer configurations.
9. Test ReLU, Tanh and Sigmoid activation functions.
10. Train each model using the training dataset.
11. Evaluate each model using test accuracy and test loss.
12. Plot training and validation accuracy and loss.
13. Give a new Iris flower measurement as input.
14. Predict its species using the trained model.
15. Display the predicted class.

## PROGRAM

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
from sklearn.datasets import load_iris
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler, OneHotEncoder
```

```
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import Dense, Input
```

```
# Load Iris dataset
```

```
iris = load_iris()
```

```
X = iris.data
```

```
y = iris.target.reshape(-1, 1)
```

```
classes = iris.target_names
```

```
# One-hot encoding
```

```
encoder = OneHotEncoder(sparse_output=False)
```

```
y_encoded = encoder.fit_transform(y)
```

```
# Standardize input features
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

```
# Split dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
X_scaled,  
y_encoded,  
test_size=0.2,  
random_state=42  
)
```

**# Function to create MLP**

```
def create_mlp(input_dim, output_dim, hidden_layers, activation):
```

```
    model = Sequential()
```

```
    model.add(Input(shape=(input_dim,)))
```

```
    for units in hidden_layers:
```

```
        model.add(Dense(units, activation=activation))
```

```
    model.add(Dense(output_dim, activation='softmax'))
```

```
    model.compile(
```

```
        optimizer='adam',
```

```
        loss='categorical_crossentropy',
```

```
        metrics=['accuracy']
```

```
    )
```

```
return model
```

```
# Different hidden layer configurations
```

```
hidden_layer_configs = [
```

```
    [8],
```

```
    [16, 8],
```

```
    [32, 16, 8]
```

```
]
```

```
# Different activation functions
```

```
activations = [
```

```
    'relu',
```

```
    'tanh',
```

```
    'sigmoid'
```

```
]
```

```
results = []
```

```
last_model = None
```

```
last_history = None
```

```
# Train and evaluate different MLP configurations
```

**for hidden\_layers in hidden\_layer\_configs:**

**for activation in activations:**

**print("\n-----")**

**print("Hidden Layers:", hidden\_layers)**

**print("Activation:", activation)**

**print("-----")**

**model = create\_mlp(**

**input\_dim=4,**

**output\_dim=3,**

**hidden\_layers=hidden\_layers,**

**activation=activation**

**)**

**history = model.fit(**

**X\_train,**

**y\_train,**

**epochs=50,**

**batch\_size=5,**

**verbose=0,**

**validation\_split=0.1**

**)**

```
test_loss, test_acc = model.evaluate(  
    X_test,  
    y_test,  
    verbose=0  
)  
  
print("Test Accuracy:", round(test_acc, 4))  
print("Test Loss:", round(test_loss, 4))  
  
results.append([  
    hidden_layers,  
    activation,  
    test_acc,  
    test_loss  
)  
  
last_model = model  
last_history = history
```

**# Display results**

```
print("\n\nMLP PERFORMANCE RESULTS")  
print("-----")
```

**for result in results:**

```
print(  
    "Hidden Layers:", result[0],  
    "| Activation:", result[1],  
    "| Accuracy:", round(result[2], 4),  
    "| Loss:", round(result[3], 4)  
)
```

```
# Plot accuracy
```

```
plt.figure(figsize=(10, 5))
```

```
plt.plot(  
    last_history.history['accuracy'],  
    label='Training Accuracy'  
)
```

```
plt.plot(  
    last_history.history['val_accuracy'],  
    label='Validation Accuracy'  
)
```

```
plt.xlabel('Epoch')
```

```
plt.ylabel('Accuracy')
```

```
plt.title('MLP Training and Validation Accuracy')
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

```
# Plot loss
```

```
plt.figure(figsize=(10, 5))
```

```
plt.plot(  
    last_history.history['loss'],  
    label='Training Loss'  
)
```

```
plt.plot(  
    last_history.history['val_loss'],  
    label='Validation Loss'  
)
```

```
plt.xlabel('Epoch')
```

```
plt.ylabel('Loss')
```

```
plt.title('MLP Training and Validation Loss')
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

```
# New Iris flower prediction
```

```
sepal_length = 5.1
```

```
sepal_width = 3.5
```

```
petal_length = 1.4
```

```
petal_width = 0.2
```

```
user_input = np.array([
```

```
    [sepal_length, sepal_width, petal_length, petal_width]
```

```
])
```

```
# Scale the new input
```

```
user_input_scaled = scaler.transform(user_input)
```

```
# Predict using the last trained model
```

```
prediction = last_model.predict(
```

```
    user_input_scaled,
```

```
    verbose=0
```

```
)
```

```
predicted_class_index = np.argmax(prediction)
```

```
predicted_class_name = classes[predicted_class_index]
```

```
print("\nNEW IRIS FLOWER PREDICTION")
```

```
print("-----")
```

```
print("Sepal Length:", sepal_length)
```

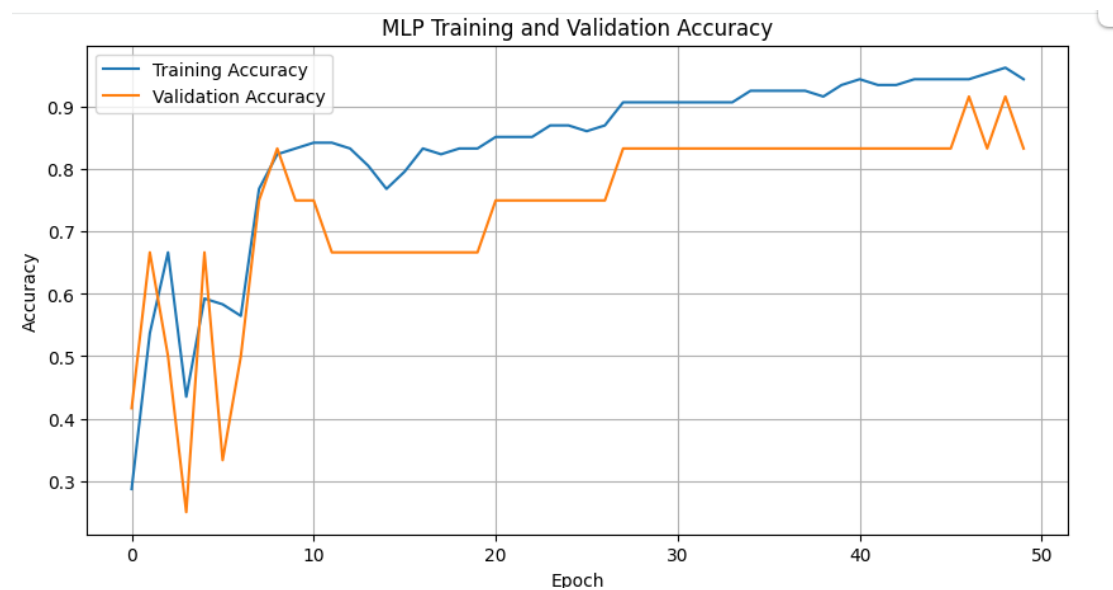
```
print("Sepal Width :", sepal_width)
```

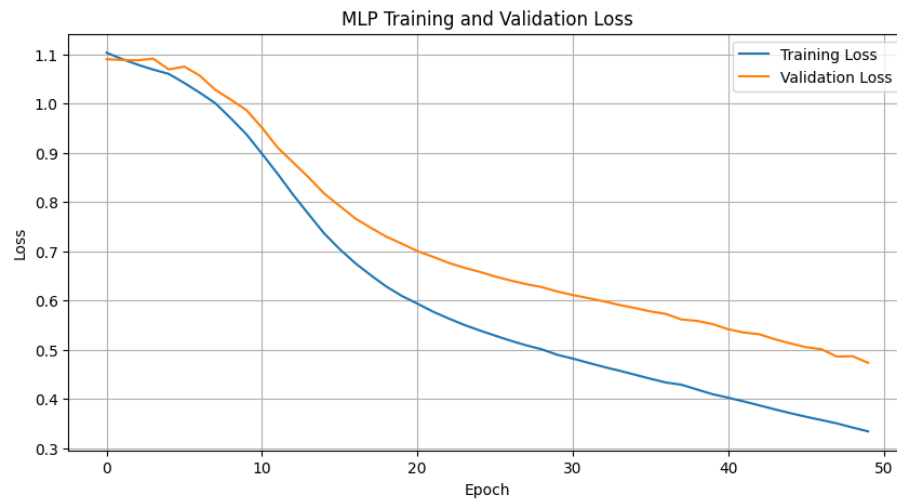
```
print("Petal Length:", petal_length)
```

```
print("Petal Width :", petal_width)
```

```
print("\nPredicted Iris Species:", predicted_class_name)
```

## OUTPUT





NEW IRIS FLOWER PREDICTION

Sepal Length: 5.1  
Sepal Width : 3.5  
Petal Length: 1.4  
Petal Width : 0.2

Predicted Iris Species: setosa

## RESULT

The Multi-Layer Perceptron was successfully implemented using TensorFlow/Keras for classification of the Iris dataset. Different hidden layer configurations and activation functions were tested, and their performance was evaluated using accuracy and loss. The trained MLP was also used to successfully predict the species of a new Iris flower.